**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

**Факультет «Инженерный бизнес и менеджмент»**
**Кафедра «Промышленная логистика»**

# ОТЧЕТ
## по лабораторной работе №3
## по дисциплине «Разработка пользовательских веб-интерфейсов»

Тема: «Знакомство с Node.js, npm. Вёрстка интерфейса с карточками. Двухстраничное приложение с использованием mock-данных.».

Вариант: продукты, компонент Bootstrap: значки (badge).

Выполнила:
студентка группы ИБМ3-54Б
Горбунова В.С.

Москва, 2026 г.

## 1. Цель работы

Знакомство с Node.js, npm, написание простого приложения на JavaScript. Изучение инструментов построения пользовательских интерфейсов веб-сайтов: HTML, CSS, JavaScript, Bootstrap. Освоение работы с DOM, модульной структурой проекта, mock-данными и переиспользуемыми компонентами.

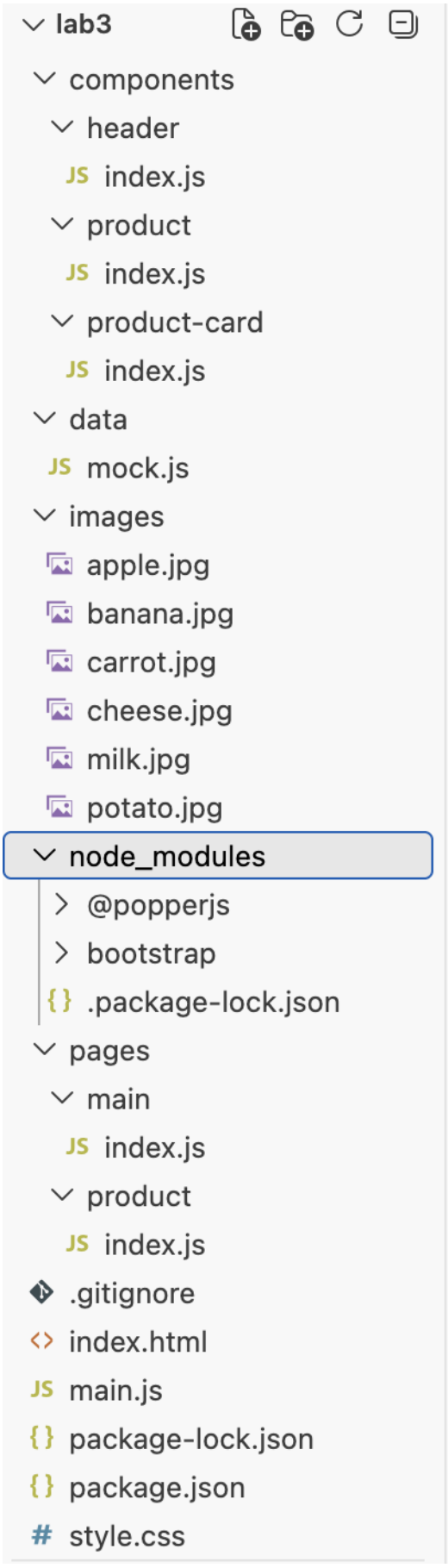
## 2. Инициализация проекта

Проект инициализирован командами в терминале VS Code:

```
npm init -y
npm i bootstrap
```

В результате созданы файлы package.json (описание проекта и зависимостей) и package-lock.json (дерево зависимостей). Установлен пакет Bootstrap, появилась папка node_modules. Также создан файл .gitignore со строкой node_modules/, чтобы не включать громоздкие сторонние библиотеки в удаленный репозиторий.

## 3. Структура проекта


lab3
components
header
index.js
product
index.js
product-card
index.js
data
mock.js
images
apple.jpg
banana.jpg
carrot.jpg
cheese.jpg
milk.jpg
potato.jpg
node_modules
@popperjs
bootstrap
.package-lock.json
pages
main
index.js
product
index.js
.gitignore
index.html
main.js
package-lock.json
package.json
style.css

```
lab3/
├── node_modules/
├── .gitignore
├── package.json
├── package-lock.json
├── index.html
├── main.js
├── style.css
├── data/
│   └── mock.js
├── pages/
│   ├── main/
│   │   └── index.js
│   └── product/
│       └── index.js
└── components/
    ├── header/
    │   └── index.js
    ├── product-card/
    │   └── index.js
    └── product/
        └── index.js
```

Назначение ключевых файлов и директорий проекта:

**index.html** — единственная HTML-страница, содержит корневой контейнер , подключение стилей Bootstrap и файла main.js в качестве исполняемого архитектурного модуля.

**main.js** — главная точка входа клиентской логики, создаёт управляющий объект MainPage и вызывает его метод рендеринга.

**data/mock.js** — изолированная mock-коллекция товарных позиций для эмуляции ответов серверной части.

**pages/** — папка страниц приложения, представляющих собой крупные самостоятельные интерфейсные блоки.

**components/** — изолированные переиспользуемые атомарные компоненты графического интерфейса.

**style.css** — каскадные таблицы стилей, реализующие визуальные правила дизайн-системы Яндекса.

## 4. Компоненты приложения

Приложение спроектировано на базе принципов компонентного подхода с жестким разграничением зон ответственности между страницами и составными виджетами.

### 4.1. Страницы (pages/)

**MainPage** (pages/main/index.js) — страница списка товаров. Отвечает за сборку и отображение каталога. Включает шапку с кнопкой «Домой» и сквозным счётчиком товаров, селект-фильтр по категориям продуктов (с выводом количества доступных позиций в каждой из них), интерактивную кнопку «+ Добавить / вернуть карточку», а также грид-контейнер с карточками товаров, отрисованными на основе mock-данных.

**ProductPage** (pages/product/index.js) — страница подробностей товара. Включает общую шапку приложения с кнопкой «Домой» и развернутую крупную карточку выбранного продукта с детализированным текстовым описанием.

### 4.2. Компоненты (components/)

**HeaderComponent** — шапка приложения. Повторно используется на обеих страницах, принимает внешний колбэк для кнопки возврата «Домой» и текущее количество товаров для динамического отображения внутри счётчика-значка.

**ProductCardComponent** — интерактивная карточка товара для страницы списка каталога. Отображает изображение, наименование, краткое описание,

стоимость, метки категории и цены в виде встроенных значков, а также содержит кнопки действий «Подробнее» и «Удалить».

**ProductComponent** — расширенная карточка товара для страницы подробностей. Применяет горизонтальную разметку с крупным изображением и развёрнутым описанием, дополненным соответствующими значками категории и стоимости.

### 4.3. Шаблон класса компонента

Каждый структурный компонент реализован как независимый ES-модуль, экспортирующий класс со стандартным набором методов:

**constructor(parent)** — конструктор класса, сохраняющий ссылку на родительский DOM-элемент;

**getHTML(data)** — метод, возвращающий строку шаблона HTML-разметки;

**addListeners(...)** — метод, навешивающий необходимые обработчики событий на интерактивные элементы;

**render(...)** — метод управления рендером, осуществляющий вставку разметки в родительский элемент и вызывающий addListeners. Такая структура делает компоненты слабосвязанными, изолированными и повторно используемыми.

## 5. Mock-данные

Файл data/mock.js экспортирует массив mockProducts, состоящий из 6 объектов. Каждый объект описывает конкретную товарную позицию категории продуктов питания и содержит поля: id (уникальный идентификатор), src (путь к изображению), title (название), text (детальное описание), category (категория: fruit, vegetable, dairy) и price (числовое значение стоимости). Использование mock-коллекции позволяет полноценно отлаживать интерфейс без развертывания

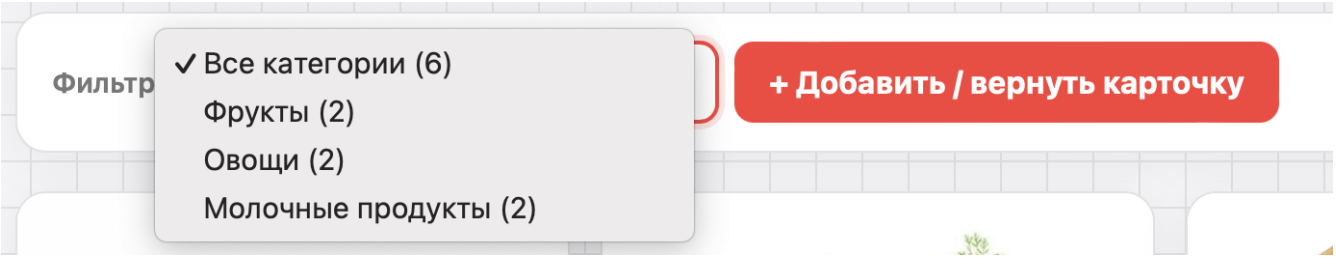
серверной части. В будущем данный модуль может быть прозрачно заменен на fetch()-запрос к реальному удаленному API без переписывания логики компонентов.

## 6. Реализация функциональности

### 6.1. Отображение списка товаров

При первичной или повторной загрузке страницы скрипт main.js создает экземпляр класса MainPage и вызывает метод render(). Страница вставляет собственный базовый HTML-шаблон в корневой контейнер, после чего итерационно обходит массив данных и генерирует по одной карточке ProductCardComponent на каждый товар из отфильтрованного списка.

### 6.2. Фильтрация




Селект фильтра содержит четыре опции: «Все категории», «Фрукты», «Овощи», «Молочные продукты». При изменении выбранного значения срабатывает обработчик clickFilter, сохраняющий категорию в локальное состояние (this.filter) и инициирующий метод перерисовки страницы this.render(). Внутренний метод getData() возвращает либо весь массив, либо отфильтрованный срез. В подписях опций динамически выводится текущее количество доступных товаров в каждой категории.

### 6.3. Переход на страницу товара




При клике на кнопку «Подробнее» обработчик clickCard считывает уникальный data-id кнопки, инициализирует новый экземпляр ProductPage с этим идентификатором и вызывает его метод отрисовки. Старый HTML-код затирается полной очисткой родительского контейнера (this.parent.innerHTML = ''), после чего рендерится новая страница подробного обзора.

### 6.4. Удаление карточки




При клике на кнопку удаления («Удалить») обработчик clickDelete считывает data-id кнопки, находит индекс целевого товара в массиве mockProducts через метод findIndex, вырезает элемент методом splice и сохраняет удалённый объект в стек-историю deletedProducts (для возможности последующего восстановления), после чего вызывает повторный рендер текущей страницы.

### 6.5. Добавление / возврат карточки




Интерактивная кнопка «+ Добавить / вернуть карточку» функционирует в двух режимах:

**Если массив deletedProducts непуст:** из него методом pop() извлекается последний удалённый объект и возвращается в массив mockProducts (принцип LIFO);

**Если история удалений пуста:** создается полная копия первого базового товара с новым уникальным ID (вычисляемым через reduce как максимальный существующий ID + 1) и добавляется в конец массива. Далее вызывается итоговый render().

### 6.6. Кнопка «Домой»




На обеих страницах приложения в шапке присутствует фиксированная кнопка «Домой». Её зарегистрированный обработчик создает экземпляр MainPage и вызывает метод render(), возвращая пользователя к исходному списку товаров с любого экрана без перезагрузки вкладки.

## 7. Использование компонента «значки» (badge)

Согласно выданному варианту задания (тема — продукты, компонент — значки), элементы Bootstrap badge применены в четырёх различных сценариях интерфейса:

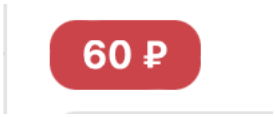
### 7.1. Значок категории на карточке товара




```
<span class="badge text-bg-success">Фрукт</span>
```

Цветовая палитра значка динамически привязана к категории продукта: класс text-bg-success используется для фруктов, text-bg-warning — для овощей, а text-bg-info — для молочных продуктов.

### 7.2. Значок цены на карточке товара




```
<span class="badge text-bg-danger fs-6">60 ₽</span>
```

Вспомогательный типографический класс fs-6 увеличивает размер шрифта внутри значка, обеспечивая высокую читаемость и визуальный акцент на стоимости продукта.

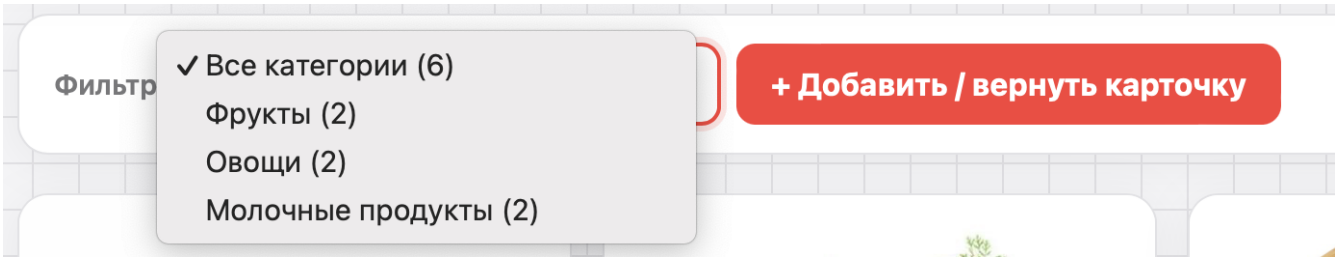
### 7.3. Значок-счётчик в шапке




```
<span class="badge text-bg-primary fs-6">Всего: 6</span>
```

Отображает суммарное число товаров в текущей коллекции. Значение полностью интерактивно и обновляется при каждом цикле рендеринга — при удалении или добавлении карточек счётчик мгновенно меняет свое значение.

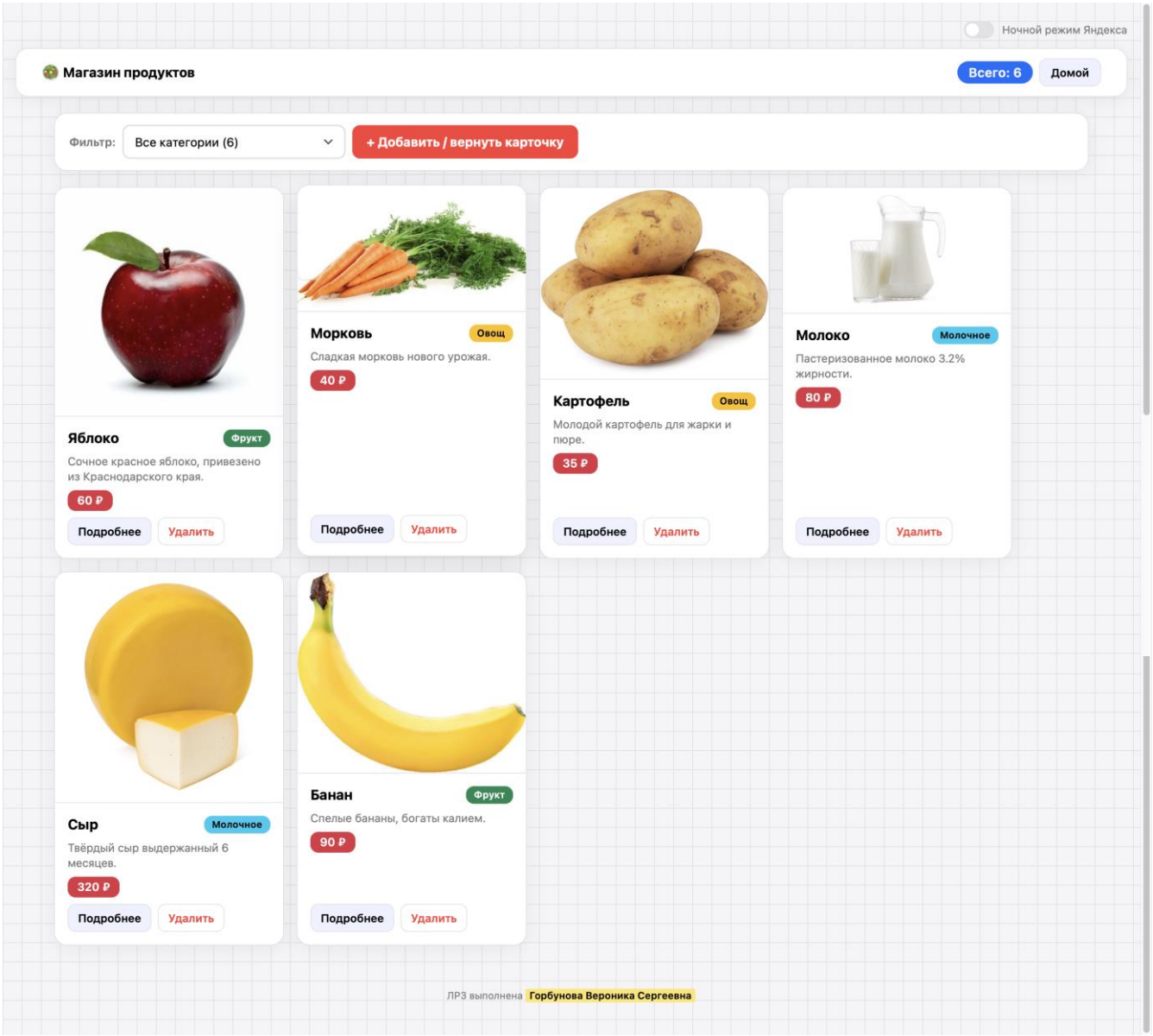
### 7.4. Счётчики в фильтре




В названиях опций селекта выводится количество товаров каждой категории: «Все категории (6)», «Фрукты (2)», «Овощи (2)», «Молочные продукты (2)», что позволяет пользователю видеть распределение продуктов до применения

фильтра. Значки badge реализуют одновременно три сценария из документации Bootstrap: ярлыки-метки, вывод значений и счетчики количества.

## 8. Оформление




Внешний вид разработанного интерфейса соответствует дизайн-системе Яндекса. Внедрены нативные CSS-переменные для управления цветами, тенями и радиусами скруглений. Использована минималистичная палитра: светлый серый фон с фирменным сетчатым паттерном, контрастные белые карточки и акцентный алый цвет элементов управления. Все блоки снабжены мягкими тенями.

Интегрирован полноценный ночной режим, переключаемый тумблером вверху страницы через селектор :has(#theme-switch:checked), динамически меняющий значения CSS-переменных темы. Готовые стили Bootstrap (сетка, карточки, кнопки, значки) гармонично дополнены собственным файлом style.css.